

TripSage AI

Multi-Agent Travel Planner for South India

Complete Software Design Document — 3-Day Build Plan

Prepared for: AI/ML Campus Placement Portfolio

Stack: FastAPI · LangGraph · ChromaDB · SQLite · Groq/Ollama · Streamlit

1. Executive Summary

TripSage AI is a multi-agent travel planning system that generates complete, personalized, day-wise itineraries for trips across six South Indian states. Users select origin, destination, trip length, budget, transport preference, and interests — the system then plans the entire trip: travel mode, hotels, food, attractions, and a realistic budget.

This is a hybrid system by design: some agents reason with an LLM, others are pure deterministic code. This mirrors how real AI products are built — not every step needs an LLM call, and knowing when to skip one is itself a production engineering decision.

Scoped tightly for a 3-day build: 5 agents (not 8), one external API, one static dataset, Streamlit instead of React. Every cut is intentional and explained below so you can defend it in an interview.

2. Problem Statement

Planning a multi-day trip manually means juggling six separate searches: transport options, hotel comparisons, restaurant lists, attraction research, day-sequencing, and budget tracking — usually across six browser tabs.

- Generic travel chatbots give vague, ungrounded suggestions with no real place data behind them
- Budget estimates are usually missing or wildly inaccurate
- Day-wise sequencing (what to do on Day 1 vs Day 2) is the most tedious part and rarely automated well

TripSage AI solves this with a grounded multi-agent pipeline: real place data (via RAG), real distance/cost data (via structured lookup), and an LLM that composes and reviews the final plan rather than inventing facts.

3. Why This Problem Matters

- Travel-tech is a proven, easy-to-explain domain — every interviewer immediately understands the use case, unlike niche enterprise tools
 - Multi-agent orchestration with LangGraph is one of the most asked-about patterns in 2025–2026 AI engineering interviews
 - Demonstrates a key senior-level skill: deciding which steps need an LLM and which don't, instead of LLM-ifying everything
 - Combines RAG + structured data + agents in one project — three major interview topics in a single, demoable system
-

4. Target Users

User	Context	Use Case
College student	Budget trip with friends	Quick, cheap multi-day itinerary
Working professional	Weekend getaway	Fast plan without research effort
Family traveler	Festival / holiday trip	Day-wise plan with food and rest stops

5. Functional Requirements

- **User selects state** → dependent city dropdown populates (origin and destination)FR1:
- **User sets trip days, budget (optional), transport preference, and interests (multi-select)**FR2:
- **Planner Agent converts the form input into a structured task plan (JSON)**FR3:
- **Knowledge Retriever performs semantic search over ChromaDB filtered by city + interest tags**FR4:
- **Transport+Budget module looks up distance, duration, and cost from a static SQLite table — no LLM call**FR5:
- **Itinerary Composer Agent synthesizes retrieved data into a day-wise schedule**FR6:
- **Reviewer Agent validates the itinerary against day-count and budget constraints, and fixes inconsistencies**FR7:
- **Final itinerary returned as structured JSON and rendered in Streamlit**FR8:
- **Guardrail rejects unsupported cities/states with a clear message instead of hallucinating a plan**FR9:

6. Non-Functional Requirements

Requirement	Target
End-to-end plan generation time	< 15 seconds
Supported cities at launch	15–20 across 6 states
LLM provider	Groq free tier (fast inference) or local Ollama
Deployment	Free tier only (Hugging Face Spaces / Render)
Reviewer correction rate	Catches at least missing-day or over-budget errors

7. Complete Tech Stack & Justification

Core Backend

Technology	Purpose	Why chosen / Alternatives rejected
FastAPI	REST API wrapping the LangGraph workflow	Async-native, you already know it. Flask rejected — no native async, slower for concurrent agent calls.
LangGraph	Agent orchestration / state machine	Deterministic node sequencing with shared state, easy to debug. Plain LangChain agent executor rejected — non-deterministic tool-calling loops are harder to control and debug in 3 days.
LangChain	LLM call wrappers, prompt templates	Works cleanly inside LangGraph nodes, huge community support.
Groq API (Llama 3.1 8B/70B)	LLM inference	Free tier, extremely fast inference (good for demo latency). Ollama kept as local fallback — no GPU needed for Groq, critical since most student laptops can't run local 8B fast.
ChromaDB	Vector store for unstructured place data	Runs embedded, zero setup, Python-native. Pinecone/Weaviate rejected — unnecessary network hop and setup cost for 3 days.
sentence-transformers (all-MiniLM-L6-v2)	Embedding model	Free, local, fast enough for ~500-1000 chunks. OpenAI embeddings rejected — adds cost and an API dependency.
SQLite	Structured data: distances, transport costs, hotel price bands	Zero setup, file-based, real SQL — better resume signal than a JSON dict, costs no extra time. Postgres rejected — needs a running server, unnecessary for this scale.

Frontend

Technology	Purpose	Why
Streamlit	UI	Builds full interactive UI in hours, not days. React rejected for THIS project specifically — the time saved (≈1 full day of API/CORS/state plumbing) goes directly into agent quality instead. Defend this choice directly in interviews: it's a deliberate trade-off, not a skill gap.

Data Sources

Source	Used for	Why chosen
Wikivoyage (one-time scrape)	Attraction descriptions, neighborhood info, practical tips	Already organized by city, written for travelers, free, no API key, no rate limits to worry about mid-project.
OpenTripMap API	Hotels and restaurants (POI data with ratings, coordinates)	Free tier, structured POI categories, no billing setup unlike Google Places.
Wikipedia API	Backup/fallback for cities with thin Wikivoyage coverage	Free, no key, good fallback so retrieval never returns empty for a supported city.

Source	Used for	Why chosen
Static distance/cost matrix (self-curated)	Transport durations and approximate costs between city pairs	No transport API has a usable free tier (IRCTC/flight APIs require paid access). For ~20-30 city pairs, manually curating once is faster and more reliable than fighting a flaky free API for 2 days.

8. Folder Structure

```

tripsage-ai/
├── backend/
│   ├── app/
│   │   ├── main.py                # FastAPI endpoint
│   │   ├── api/routes/
│   │   │   ├── plan_trip.py       # POST /plan-trip
│   │   │   └── health.py
│   │   ├── agents/
│   │   │   ├── state.py           # Shared LangGraph state schema
│   │   │   ├── planner_agent.py   # Agent 1: parses request
│   │   │   ├── knowledge_retriever.py # Agent 2: RAG over ChromaDB
│   │   │   ├── transport_budget.py # Agent 3: pure code, SQLite lookup
│   │   │   ├── itinerary_composer.py # Agent 4: LLM synthesis
│   │   │   └── reviewer_agent.py   # Agent 5: validation + fix
│   │   ├── graph/
│   │   │   └── workflow.py         # LangGraph workflow definition
│   │   ├── core/
│   │   │   ├── config.py
│   │   │   ├── embeddings.py      # MiniLM wrapper
│   │   │   ├── vectorstore.py     # ChromaDB client
│   │   │   ├── llm.py             # Groq/Ollama wrapper
│   │   │   ├── db.py              # SQLite connection
│   │   │   └── guardrails.py      # Unsupported city/state check
│   │   ├── models/
│   │   │   ├── request.py
│   │   │   └── response.py
│   │   └── utils/logger.py
│   ├── data/
│   │   ├── ingest_wikivoyage.py    # One-time scrape + chunk + embed
│   │   ├── ingest_opentripmap.py   # One-time hotel/restaurant fetch
│   │   ├── build_transport_db.py    # Builds SQLite distance/cost table
│   │   ├── transport_matrix.json   # Source data for SQLite build
│   │   └── chroma_store/           # Persisted ChromaDB files
│   ├── tests/
│   │   ├── test_agents.py
│   │   └── test_retriever.py

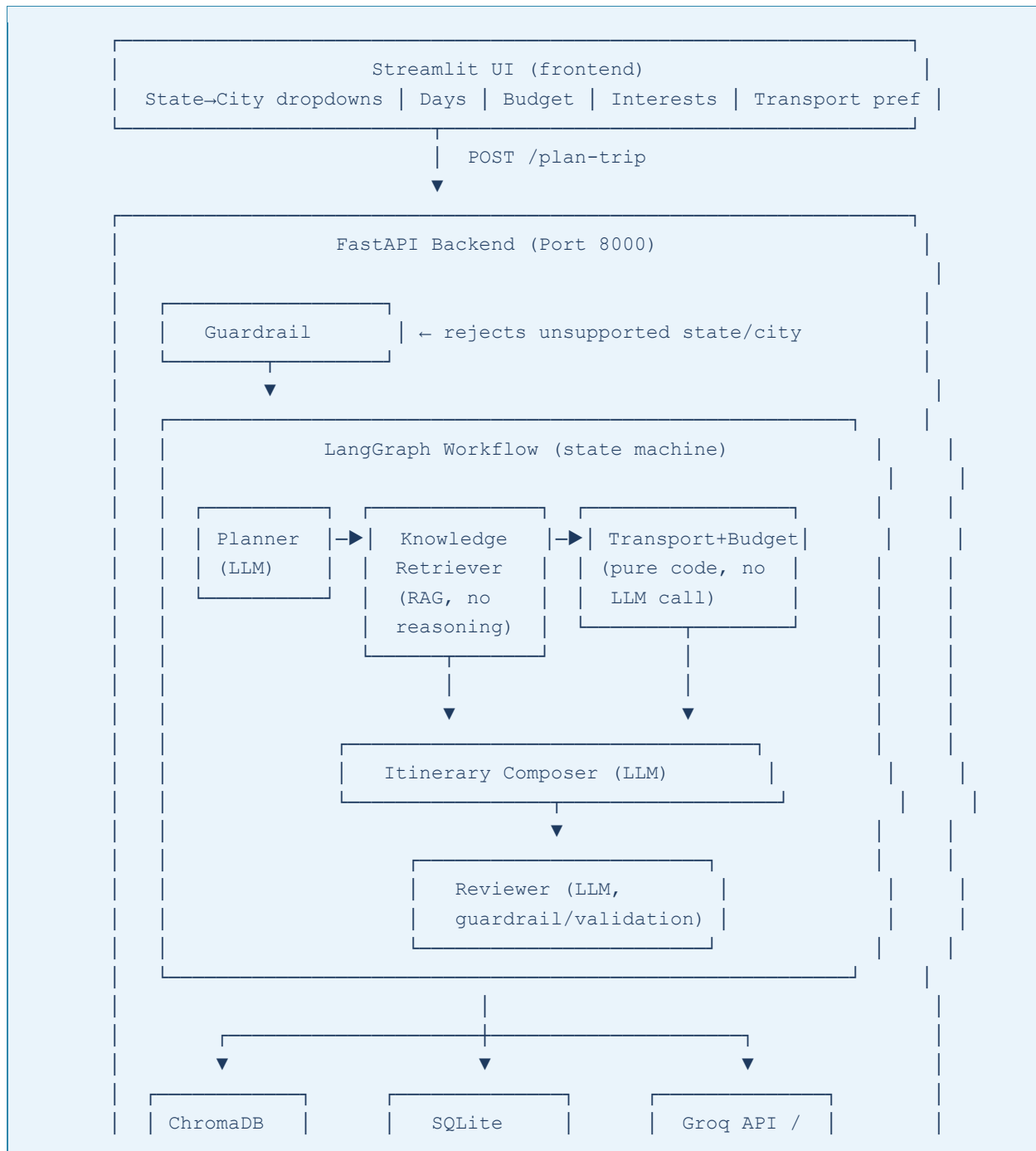
```

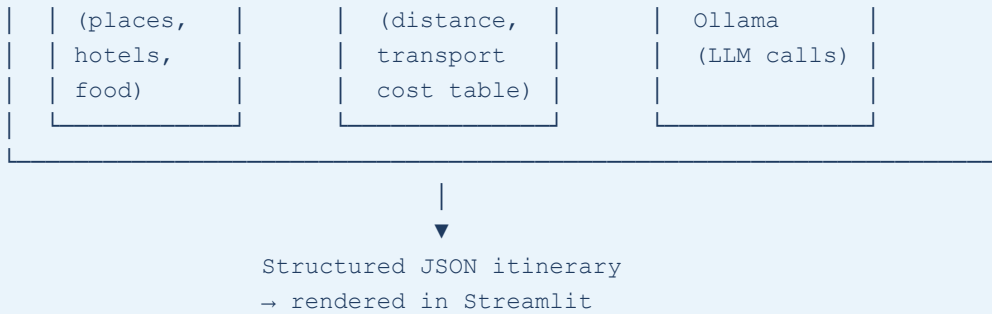
```

├── test_workflow.py
├── requirements.txt
├── frontend/
│   ├── streamlit_app.py          # Single-file Streamlit UI
│   ├── .env.example             # GROQ_API_KEY, OPENTRIPMAP_KEY
│   └── README.md

```

9. High-Level Architecture Diagram





10. LangGraph State Schema

The state object passed between all 5 agents:

```
# agents/state.py
from typing import TypedDict, Optional, List, Dict

class TripState(TypedDict):
    # Input
    origin: str
    destination: str
    days: int
    budget: Optional[int]
    transport_pref: str
    interests: List[str]

    # Agent outputs (filled progressively)
    task_plan: Optional[Dict]           # From Planner Agent
    retrieved_places: Optional[List]    # From Knowledge Retriever
    transport_info: Optional[Dict]      # From Transport+Budget
    budget_breakdown: Optional[Dict]    # From Transport+Budget
    draft_itinerary: Optional[Dict]     # From Itinerary Composer
    final_itinerary: Optional[Dict]     # From Reviewer (after fixes)

    # Control
    current_agent: str
    errors: List[str]
```

11. Agent Definitions

Agent 1 — Planner (LLM, lightweight)

Property	Detail
Input	origin, destination, days, budget, transport_pref, interests
Output	task_plan: structured breakdown of what needs to be retrieved
System prompt focus	Parse the trip request into specific retrieval queries — e.g. 'temples in Madurai', 'budget hotels near Kochi beach', 'vegetarian food in Mysore'
Output format	JSON: {retrieval_queries: [...], priority_interests: [...], day_budget_per_day: int}
Why LLM here	Translating loose interests into specific, retrievable queries genuinely benefits from language understanding.

Agent 2 — Knowledge Retriever (RAG, no LLM reasoning)

Property	Detail
Input	task_plan.retrieval_queries, destination city
Output	retrieved_places: ranked list of attractions/hotels/restaurants with metadata
How it works	Embed each query with MiniLM, run ChromaDB cosine similarity search filtered by metadata (city=destination), top-5 per category
Output format	List of {name, category, description, rating, source}
Why NO LLM call here	This is pure retrieval — adding an LLM call would only slow it down and risk paraphrasing facts incorrectly. Filtered vector search is strictly better here.

Agent 3 — Transport + Budget (pure code, zero LLM calls)

Property	Detail
Input	origin, destination, days, transport_pref, budget
Output	transport_info, budget_breakdown
How it works	SQLite query: SELECT * FROM distances WHERE origin=? AND destination=?. Applies simple cost rules: hotel_cost_per_night × days + transport_cost + food_estimate × days
Output format	{mode, duration_hours, transport_cost, hotel_estimate, food_estimate, total_estimate}
Why this agent exists as pure code	Arithmetic and table lookups should never go through an LLM — it's slower, costs tokens, and LLMs are unreliable at exact math. This is the clearest 'know when not to use an LLM' moment in the whole project — say this explicitly in interviews.

Agent 4 — Itinerary Composer (LLM)

Property	Detail
Input	retrieved_places, transport_info, budget_breakdown, days, interests
Output	draft_itinerary: day-wise schedule
System prompt focus	Sequence retrieved places into a logical day-wise plan considering proximity/category balance, respecting the day count exactly
Output format	JSON: {day_1: {morning, afternoon, evening, meals}, day_2: {...}, ...}
Why LLM here	Sequencing and balancing a schedule from raw data is a synthesis task — this is where the LLM adds real value over template logic.

Agent 5 — Reviewer (LLM, guardrail)

Property	Detail
Input	draft_itinerary, days, budget, budget_breakdown
Output	final_itinerary: validated and corrected plan
System prompt focus	Check: does the itinerary have exactly `days` entries? Does total_estimate exceed budget (if set)? Are there duplicate or impossible same-day far-apart locations?
Output format	Same schema as draft_itinerary, with a 'review_notes' field listing what was fixed
Why this agent exists	LLM-composed plans can silently drop a day or overshoot budget. A dedicated validation pass catches this before the user sees it — directly mirrors a production QA/guardrail step.

12. Detailed Data Flow

Step	What happens	Where
1	User fills Streamlit form, clicks 'Plan My Trip'	streamlit_app.py
2	POST /plan-trip sends form data as JSON	plan_trip.py
3	Guardrail checks origin/destination are in supported city list	guardrails.py
4	LangGraph workflow initialized with TripState	workflow.py
5	Planner Agent parses request into retrieval_queries	planner_agent.py
6	Knowledge Retriever runs ChromaDB semantic search per query, filtered by city	knowledge_retriever.py
7	Transport+Budget queries SQLite distance table, computes cost breakdown — no LLM call	transport_budget.py

Step	What happens	Where
8	Itinerary Composer synthesizes all retrieved + computed data into a day-wise draft	itinerary_composer.py
9	Reviewer validates day-count and budget, fixes issues, returns final_itinerary	reviewer_agent.py
10	FastAPI returns final_itinerary as JSON	plan_trip.py
11	Streamlit renders day-by-day cards with cost summary	streamlit_app.py

13. ChromaDB Schema

Collection name: south_india_places

```

id:          place_{city}_{category}_{index}
embedding:   [0.08, -0.21, ..., 0.44]    # 384-dim, MiniLM
document:    'Meenakshi Amman Temple is a historic...'
metadata: {
  city:      'Madurai',
  state:     'Tamil Nadu',
  category:  'attraction',    # attraction | hotel | restaurant
  interest_tags: ['temples', 'history'],
  rating:    4.6,
  source:    'wikivoyage'
}

```

14. SQLite Schema (Transport + Budget)

```

-- distances table
CREATE TABLE distances (
  origin TEXT,
  destination TEXT,
  mode TEXT,          -- flight | train | bus | car
  duration_hours REAL,
  cost_inr INTEGER,
  PRIMARY KEY (origin, destination, mode)
);

-- city_costs table (for budget estimation)
CREATE TABLE city_costs (
  city TEXT PRIMARY KEY,

```

```
hotel_budget_per_night INTEGER,  
hotel_midrange_per_night INTEGER,  
food_per_day_estimate INTEGER  
);
```

15. Chunking Strategy (Wikivoyage Ingestion)

Parameter	Value & Reason
Chunk unit	One chunk per attraction/POI section (not fixed token windows) — Wikivoyage is already well-segmented by place
Chunk size cap	300 tokens max per chunk — keeps retrieval precise per place
Metadata tagging	Each chunk tagged with city, state, category, interest_tags during ingestion script — this is what makes filtered retrieval possible
Interest tagging method	Simple keyword matching during ingestion (e.g. 'temple', 'fort' → tag 'history'/'temples') — no LLM needed for this one-time offline step

16. Prompt Engineering Strategy

Itinerary Composer — System Prompt

```
You are a travel itinerary planner. You will be given:  
- A list of retrieved attractions, hotels, and restaurants  
- Transport details and a budget breakdown  
- The exact number of days for the trip  
  
Create a day-wise itinerary using ONLY the provided places.  
Do not invent attractions, hotels, or restaurants not in the list.  
You MUST produce exactly {days} day entries – no more, no fewer.  
Balance morning/afternoon/evening activities by category.  
Respond ONLY in valid JSON matching the schema provided.
```

Reviewer — System Prompt

```
You are a strict itinerary reviewer. Check the draft itinerary for:  
1. Does it have EXACTLY {days} day entries? If not, fix it.  
2. Does total estimated cost exceed the user's budget (if given)?  
   If so, suggest cheaper alternatives from the same retrieved list.  
3. Are any places duplicated across days?
```

```
4. Are any two same-day locations implausibly far apart?
```

```
Output the corrected itinerary plus a 'review_notes' list explaining every fix you made. If no fixes needed, review_notes: [].
```

17. Memory Design

Memory here is intentionally minimal — this is a single-shot planning tool, not a multi-turn chat. But to make the project resume-worthy on 'Memory' as a concept, add lightweight session memory for refinement requests.

```
# Simple in-memory session store, keyed by session_id
session_store: Dict[str, TripState] = {}

# Lets user say: 'make day 2 cheaper' as a follow-up
# without re-entering the whole form — re-runs Reviewer + Composer
# only, reusing retrieved_places already in session state
```

This is a stretch feature — build it last, only if Day 3 has spare time. The core 5-agent pipeline must work first.

18. Guardrails

Guardrail	Implementation
Unsupported city/state	Check origin/destination against a fixed supported-city list before running the workflow. Reject with a clear message and the list of supported cities.
Empty retrieval results	If Knowledge Retriever returns zero places for the destination, fail fast with 'Insufficient data for this destination' instead of letting Composer hallucinate.
Day-count mismatch	Reviewer Agent enforces exact day count — hard requirement, not a suggestion
Budget overrun	Reviewer Agent flags and attempts to fix; if still over budget after one fix attempt, returns itinerary with a clear 'over budget by ₹X' warning rather than silently failing
LLM output validation	All agent JSON outputs validated against Pydantic schema before being passed to the next node; retry once on parse failure
Max workflow steps	LangGraph max_steps cap to prevent any accidental infinite loop

19. Error Handling

Error Scenario	Handling
Groq API rate limit hit	Fallback to local Ollama if configured, else return 503 with retry-after message
OpenTripMap API down (during ingestion)	Ingestion script logs and skips that city, continues with others — doesn't block the whole pipeline
SQLite has no route for a city pair	Transport agent returns a generic estimate based on straight-line distance heuristic, flags it as 'estimated' in output
Composer returns invalid JSON	Retry once with a stricter prompt reminder; if it fails again, surface a clean error to the user

20. Deployment

Component	Deployment choice & why
Streamlit frontend	Hugging Face Spaces — free, native Streamlit support, zero Docker config needed, fastest path to a public demo link
FastAPI backend	Can run inside the same HF Space (Streamlit calling local FastAPI process) for simplicity, OR deploy separately on Render free tier if you want them decoupled
LLM provider in production	Groq API — HF Spaces free tier has no GPU, so local Ollama won't work there. Groq's free tier is fast enough for live demos.
ChromaDB + SQLite	Both file-based — bundle the persisted chroma_store/ and .db file directly into the deployed repo so no separate database hosting is needed

Recommended path: keep FastAPI + Streamlit in one HF Space for the 3-day timeline. Splitting them into two services adds deployment complexity with no real benefit at this scale.

21. Interview Questions — TripSage AI

Multi-Agent Architecture

- Why does your system use 5 agents instead of one large LLM call? What did splitting it buy you?
- Two of your agents don't call an LLM at all. Walk me through why, and how you decided that.
- What is LangGraph and how does it differ from a plain LangChain agent loop?
- How does state get passed between agents? What would break if one agent corrupted the shared state?
- Could the Knowledge Retriever and Transport+Budget agents run in parallel instead of sequentially? What would you need to change?

RAG & Retrieval

- Why ChromaDB for places but SQLite for transport/cost data? Why not put everything in one database?
- How do you prevent retrieval from returning attractions in the wrong city?
- What would you do if a destination city had very few or zero indexed places?
- Explain how interest tags improve retrieval precision over plain semantic search.

Guardrails & Reliability

- What does your Reviewer Agent actually check, and why is that check necessary?
- How do you guarantee the itinerary has exactly the number of days the user requested?
- What happens if the Composer Agent invents a hotel that wasn't in the retrieved data?

Production & Trade-offs

- Why Streamlit instead of React for this project? Was that a shortcut or a deliberate choice?
 - Why Groq instead of a local LLM for deployment?
 - How would you scale this from 6 states to all of India? What's the actual bottleneck — the architecture or the data?
 - If budget was unlimited and you had 3 more weeks, what would you add first?
-

22. ATS-Friendly Resume Bullets

- Built TripSage AI, a multi-agent travel planning system using FastAPI, LangGraph, and ChromaDB that generates personalized day-wise itineraries across 6 South Indian states using a hybrid LLM-reasoning and deterministic-code agent pipeline
- Designed a 5-agent LangGraph workflow (Planner, Knowledge Retriever, Transport+Budget, Itinerary Composer, Reviewer) with shared typed state, deliberately routing arithmetic and lookup tasks to pure code instead of LLM calls to reduce latency and cost
- Implemented RAG pipeline over Wikivoyage and OpenTripMap data using sentence-transformers embeddings and ChromaDB metadata-filtered semantic search across attraction, hotel, and restaurant categories
- Built a structured SQLite data layer for transport distances and cost estimation, paired with vector search for unstructured content — demonstrating correct database selection based on data shape
- Engineered a Reviewer Agent as a guardrail layer that validates LLM-generated itineraries against day-count and budget constraints, with automatic correction logic

23. Implementation Roadmap — 3 Days

Day 1 — Data Foundation

Item	Detail
Goal	ChromaDB loaded with place data for 15-20 cities; SQLite has distance/cost tables
Concepts learned	Web scraping basics, embeddings, ChromaDB metadata filtering, SQLite schema design
Files to create	ingest_wikivoyage.py, ingest_opentripmap.py, build_transport_db.py, embeddings.py, vectorstore.py, db.py
Expected output	Query 'temples in Madurai' returns 5 relevant chunks; SELECT * FROM distances returns rows for test city pair
Common mistake	Forgetting metadata filtering on city — retrieval mixes places from different cities
Debug tip	Print retrieved results with their metadata for 3-4 test queries before moving on
Time	~7-8 hours

Day 2 — Agent Workflow

Item	Detail
Goal	Full 5-agent LangGraph pipeline runs end-to-end from terminal for one test trip
Concepts learned	LangGraph state machines, prompt engineering, mixing LLM and non-LLM nodes
Files to create	state.py, planner_agent.py, knowledge_retriever.py, transport_budget.py, itinerary_composer.py, reviewer_agent.py, workflow.py, llm.py
Expected output	Running workflow.invoke() for 'Hyderabad to Madurai, 3 days' prints a complete day-wise JSON itinerary
Common mistake	Composer Agent inventing places not in retrieved_places — fix with a stricter system prompt and explicit 'ONLY use provided list' instruction
Debug tip	Print state after every node — if Day-2 output looks wrong, you'll immediately see which agent introduced the problem
Time	~8 hours

Day 3 — API, UI, Guardrails, Deploy

Item	Detail
Goal	FastAPI endpoint working, Streamlit UI complete, guardrails in place, deployed live
Concepts learned	FastAPI route design, Streamlit forms, dependent dropdowns, deployment on HF Spaces

Item	Detail
Files to create	plan_trip.py, guardrails.py, streamlit_app.py, requirements.txt, README.md
Expected output	Live public URL where you can fill the form and get a rendered itinerary with cost breakdown
Common mistake	Dependent dropdown (state → city) not resetting properly in Streamlit when state changes — use st.session_state explicitly
Debug tip	Test the unsupported-city guardrail explicitly before considering this done — it's a common interview follow-up question
Time	~7-8 hours

24. Setup Checklist (Do This First)

- Get a free Groq API key: <https://console.groq.com>
- Install Python 3.11, pip, venv
- pip install fastapi langgraph langchain chromadb sentence-transformers streamlit groq
- Get a free OpenTripMap API key: <https://opentripmap.io>
- Pick your 15-20 launch cities now (e.g. Hyderabad, Bengaluru, Chennai, Kochi, Mysore, Madurai, Goa, Coorg, Ooty, Pondicherry, Hampi, Munnar...) — write this list down before Day 1 starts
- Create one GitHub repo: tripsage-ai
- Test Groq API call works before Day 1 ends — this unblocks everything else

TripSage AI v2 - Enhancement Guide

This supplement is intended to be used alongside your existing 16-page design document.

Advanced RAG

Hybrid Search (BM25 + Semantic), Cross-Encoder Reranker, MMR, Query Rewriting, Context Compression.

Agent Pipeline

Planner -> Query Rewriter -> Hybrid Retriever -> Re-ranker -> Transport Tool -> Budget Tool -> Composer -> Reviewer -> Reflection -> Guardrails.

Knowledge Base

Wikivoyage, Wikipedia, OpenStreetMap, SQLite, ChromaDB.

Memory

Long-term user preference memory using ChromaDB.

Evaluation

Precision, Recall, Latency, Hallucination Rate, Cost, Token Usage.

Deployment

FastAPI + Streamlit + Hugging Face Spaces/Render + Docker.

Testing

Unit, Integration, Retrieval, Agent and E2E tests.

Interview

Architecture decisions, trade-offs, scaling, LangGraph, RAG.